

100 DAYS OF CODE - 4

C PROGRAMMING | DEBUGGING & LOGIC CORRECTION

Multiplication Without Using the * Operator

1. Problem Statement

Correct the supplied C logic, algorithm, and flowchart so that two integers can be multiplied without using the multiplication (*) operator. The multiplication must be performed through repeated addition and must work correctly for positive values, negative values, and zero.

2. Objective

To develop a repeated-addition method for integer multiplication, identify the limitations in the original logic, and verify a corrected approach that handles signs and zero properly.

3. Original Algorithm

1. Declare num1, num2, prod and index as integers.
2. Read num1 and num2.
3. Initialize prod = 0 and index = 1.
4. Repeat while index <= num2:
 - prod = prod + num1
 - index = index + 1
5. Display prod.
6. Stop.

4. Problems in the Original Logic

- The loop depends directly on num2, so it works correctly only when num2 is positive.
- Negative input values are not handled correctly.
- Zero cases are not handled explicitly by the original logic.
- The sign of the final product is never determined or applied.

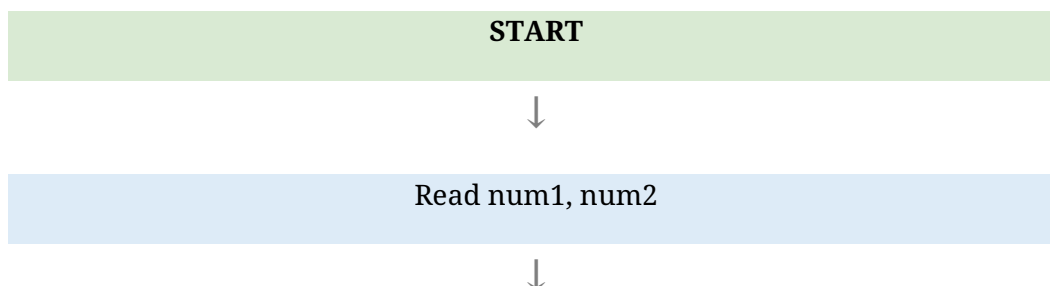
5. Original Flow of Logic

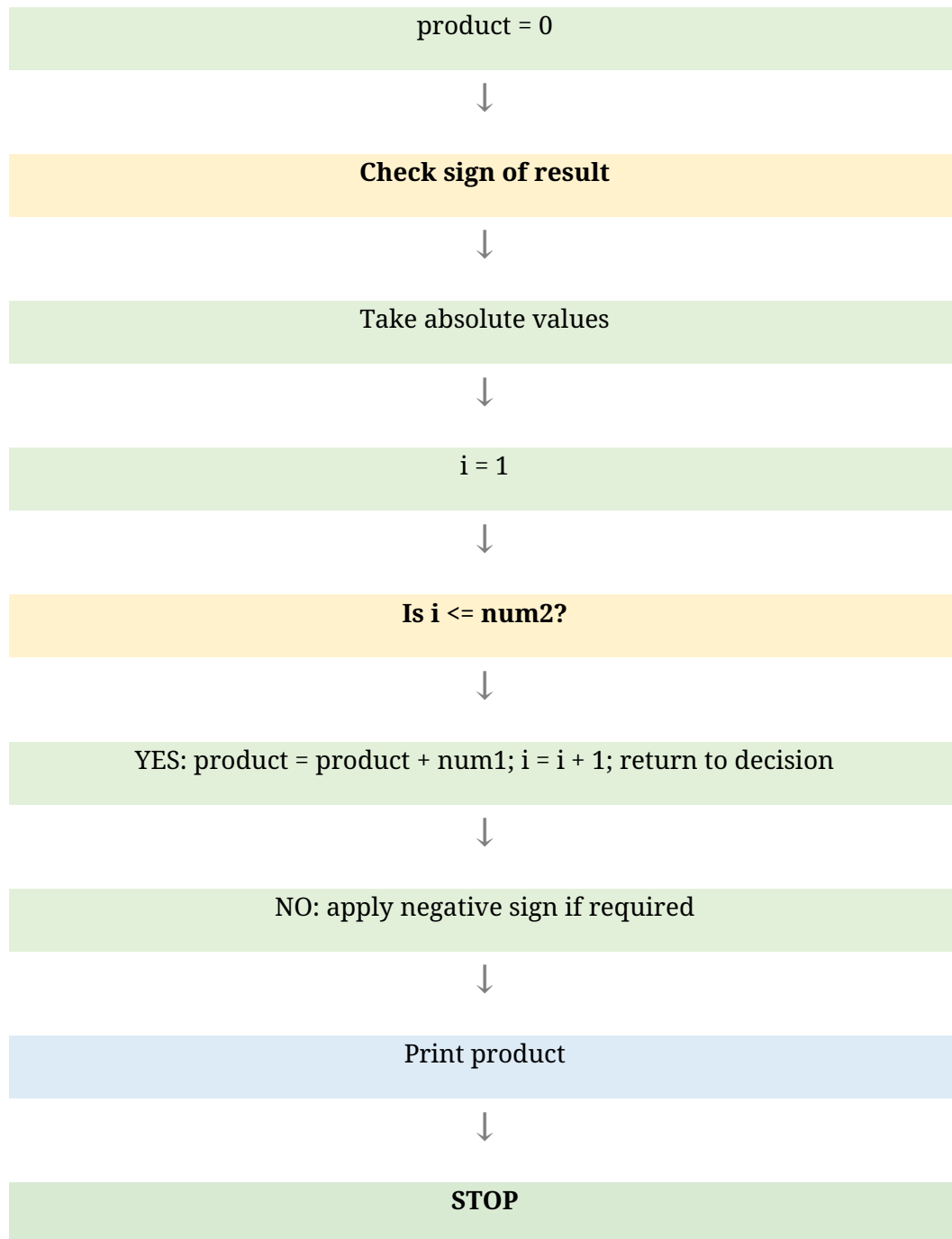
Step	Operation	Next
1	Start	2
2	Read num1, num2	3
3	prod = 0, index = 1	4
4	Is index <= num2?	Yes -> 5 / No -> 7
5	prod = prod + num1	6
6	index = index + 1	Back to 4
7	Print prod	8
8	Stop	-

6. Corrected Algorithm

1. Start.
2. Read two integers num1 and num2.
3. Initialize product = 0.
4. Determine whether the final answer should be negative.
5. Convert num1 and num2 to absolute values.
6. Repeat num2 times and add num1 to product each time.
7. If the answer should be negative, change product to -product.
8. Display product.
9. Stop.

7. Corrected Flowchart





8. Conclusion

This activity shows that multiplication can be implemented using repeated addition rather than the `*` operator. After correcting the logic to handle absolute values and the sign of the result, the method works for positive integers, negative integers, and zero.